Programming Project - Sorting Algorithms

CSE 5311 - 005 - Spring 2026

Arham Shams Sameer
1002078834

1 Introduction

For the project I implemented seven sorting algorithms from scratch in Python without using any built-in sorting functions like `sorted()` or `list.sort()`. The seven algorithms are Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Heap Sort, Quick Sort with a last-element pivot, and Quick Sort with median-of-three pivot selection. I then benchmarked all seven across different input sizes and distributions to see how they actually perform compared to what the theory says.

The code is split into four Python files:

- `sorting_algorithms.py` - the actual sort implementations
- `input_generator.py` - generates and saves test inputs
- `benchmark.py` - runs timing experiments, outputs CSV and charts
- `gui.py` - bonus Tkinter GUI for interactive comparison

All the input files are stored as plain text in the `inputs/` directory and the results (CSV + 10 PNG charts) go into `results/`.

2 Data Structures

The main data structure used throughout is Python's built-in `list`, which is a dynamic array under the hood.

I chose this because:

- Random access is $O(1)$, which is what all these sorting algorithms need
- It was the most straightforward choice for the type of data we're sorting (arrays of integers)

Every sorting function takes a list as input and returns a **new** sorted list without modifying the original. I did it this way so I could run all seven algorithms on the exact same input array in the benchmark without worrying about one algorithm messing up the data for the next one.

For the benchmark results, I used a nested dictionary structure:

```
results[distribution][algorithm][size] = seconds.
```

I originally tried storing results as a flat list of tuples but it was annoying to query later when generating the charts, so I switched to the nested dict.

3 Algorithms

3.1 Bubble Sort

Walks through the list repeatedly, swapping adjacent elements if they are out of order. After each pass the largest unsorted element ends up in its final position at the end. I added an early-exit flag (the `swapped` variable) so that if a full pass makes zero swaps, we know the list is already sorted and can stop. Without this, bubble sort was painfully slow on sorted input.

Complexity: $O(n^2)$ average and worst case, $O(n)$ best case with the early exit.

3.2 Selection Sort

Finds the minimum element in the unsorted portion and swaps it into the correct position at the front. Does this $n - 1$ times. It always does $O(n^2)$ comparisons regardless of the input distribution, which showed up clearly in the benchmarks, selection sort took roughly the same time on random, sorted, and reverse inputs.

Complexity: $O(n^2)$ in all cases.

3.3 Insertion Sort

Builds up a sorted portion of the array from left to right. For each new element, it slides it backwards until it finds the right spot. Really fast on nearly-sorted input because each element only needs to move a few positions.

Complexity: $O(n^2)$ average/worst, $O(n)$ best case.

3.4 Merge Sort

Recursively split the array in half, sort each half, then merge them back together. I first tried a version that returned new lists at every level of recursion, but it was creating way too many temporary lists and the index math got confusing. So I switched to an in-place version that uses a helper function with left/right indices and merges into the original array.

Complexity: $O(n \log n)$ in all cases, but uses $O(n)$ extra space for the merge step.

3.5 Heap Sort

Builds a max-heap in-place and then repeatedly extracts the maximum. I first tried building the heap top-down (inserting elements one by one and sifting up) but the bottom-up approach starting from the last non-leaf node is more efficient. The `_heapify` function handles the sift-down operation.

Complexity: $O(n \log n)$ in all cases. No extra space needed.

3.6 Quick Sort (Regular)

Uses the Lomuto partition scheme with the last element as the pivot. I originally had the first element as pivot but switched to last because the textbook examples used that and it was easier to follow. The problem with this version is that sorted or reverse-sorted input causes $O(n^2)$ behavior because the pivot is always the min or max.

Complexity: $O(n \log n)$ average, $O(n^2)$ worst case.

3.7 Quick Sort (Median-of-Three)

Same as above but picks the median of `a[low]`, `a[mid]`, and `a[high]` as the pivot. This avoids the $O(n^2)$ worst case on sorted and reverse-sorted inputs. I got the median-of-three selection wrong at first because I was returning the *value* instead of the *index*, which broke the partition. Once I fixed that it worked fine.

Complexity: $O(n \log n)$ average and practically always.

4 Experimental Methodology

4.1 Setup

I ran all benchmarks on my desktop (AMD Ryzen 7 7800X3D, 64GB RAM) using Python 3.12.8. Timing was done with `time.perf_counter()` because `time.time()` wasn't precise enough, the fast algorithms were showing up as 0.0 seconds.

4.2 Input Generation

I tested four input distributions:

1. **Random** - uniformly random integers in $[0, 1{,}000{,}000]$, seeded (42) for reproducibility
2. **Sorted** - already sorted: $0, 1, 2, \ldots, n-1$
3. **Reverse** - reverse sorted: $n-1, n-2, \ldots, 0$
4. **Nearly sorted** - sorted with ~5% random adjacent swaps

Each distribution was generated at six sizes: $n = 100, 500, 1000, 2000, 5000, 10000$, giving 24 input files total. All inputs are saved to files (one integer per line) so they can be reloaded and reused.

4.3 Verification

After every single benchmark run, I verify that the output matches Python's built-in `sorted()` to make sure the algorithm is actually correct. If any algorithm produces wrong output, the program crashes immediately.

5 Experimental Results

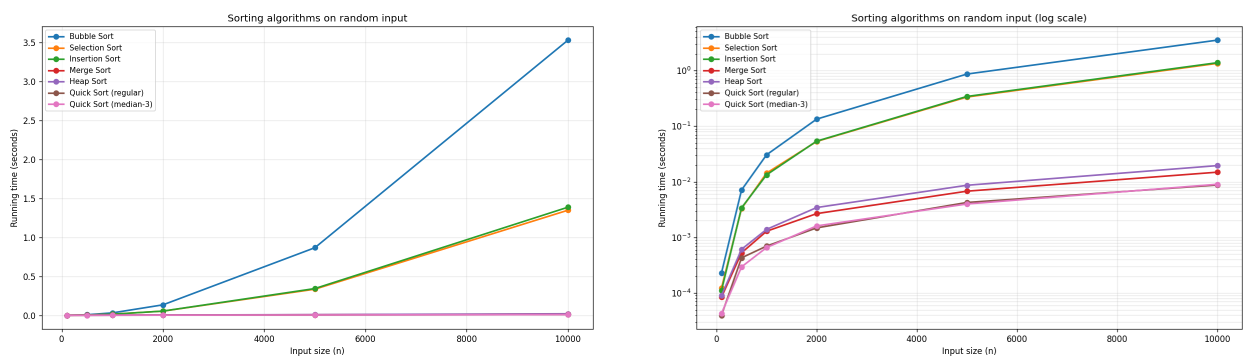
5.1 Random Input

Table 1 shows the timing results on random input.

Table 1: Running time (seconds) on random input

Algorithm	100	500	1000	2000	5000	10000
Bubble Sort	0.0002	0.0071	0.0306	0.1341	0.8666	3.5295
Selection Sort	0.0001	0.0033	0.0144	0.0532	0.3345	1.3478
Insertion Sort	0.0001	0.0034	0.0134	0.0539	0.3427	1.3864
Merge Sort	0.0001	0.0005	0.0013	0.0027	0.0068	0.0149
Heap Sort	0.0001	0.0006	0.0014	0.0034	0.0087	0.0196
Quick Sort (regular)	0.0000	0.0004	0.0007	0.0015	0.0043	0.0088
Quick Sort (median-3)	0.0000	0.0003	0.0007	0.0016	0.0040	0.0090

At $n = 10{,}000$, the $O(n^2)$ algorithms take between 1.3 and 3.5 seconds while the $O(n \log n)$ ones finish in under 20 milliseconds. That is a roughly 200x difference, which lines up with the theoretical gap between n^2 and $n \log n$ at this scale.



(a) Linear scale

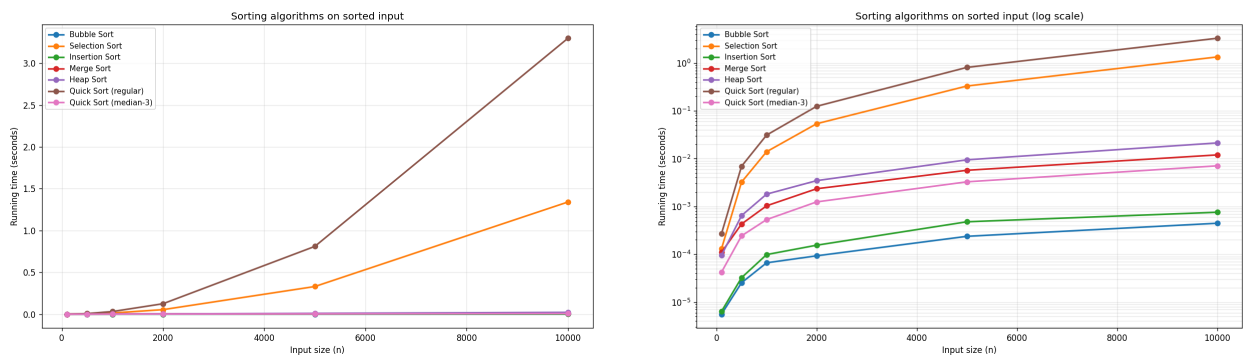
(b) Log scale

Figure 1: All algorithms on random input

5.2 Sorted Input

On sorted input:

- Bubble Sort with the early-exit flag becomes $O(n)$ and finishes in under 0.5ms even at $n = 10{,}000$
- Insertion Sort is also $O(n)$ because nothing needs to move
- Regular Quick Sort degrades to $O(n^2)$ because the last element is always the largest, giving the worst possible pivot every time
- Selection Sort doesn't care, it's $O(n^2)$ regardless

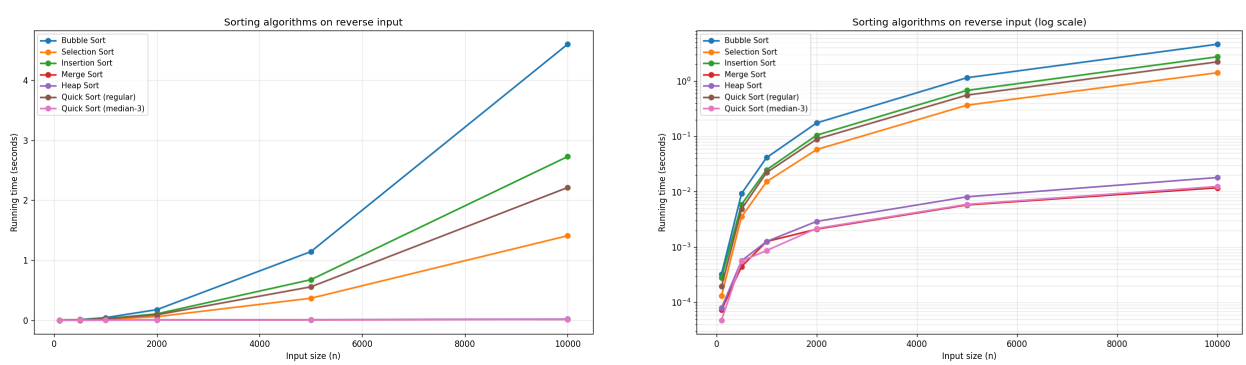


(a) Linear scale (b) Log scale

Figure 2: All algorithms on sorted input

5.3 Reverse-Sorted Input

Reverse input is basically the worst case for bubble sort (every pair needs to be swapped) and also bad for regular quicksort. The $O(n \log n)$ algorithms don't really care about the distribution.

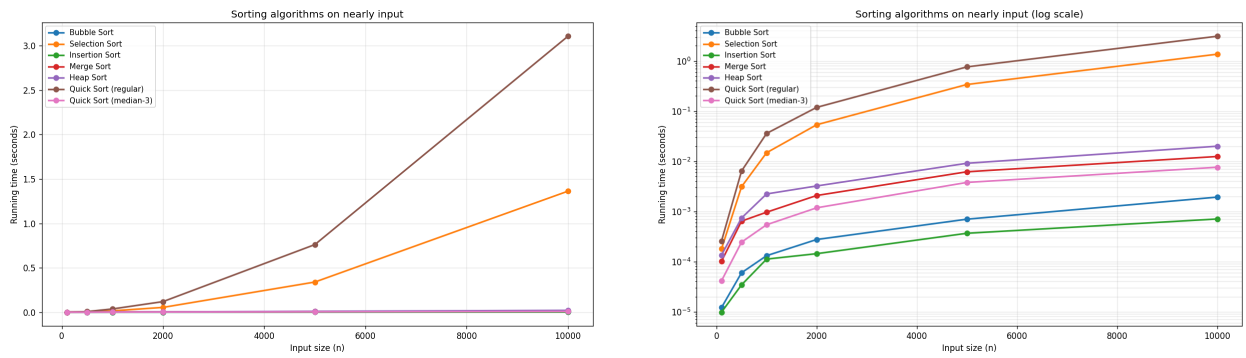


(a) Linear scale (b) Log scale

Figure 3: All algorithms on reverse-sorted input

5.4 Nearly-Sorted Input

Nearly-sorted input is where insertion sort and bubble sort work well. Since most elements are already in the right place, they only need to do a small number of swaps. Meanwhile, regular quicksort still hits $O(n^2)$ because the nearly-sorted input creates bad pivots.



(a) Linear scale (b) Log scale

Figure 4: All algorithms on nearly-sorted input

5.5 Fast Algorithms Comparison

Figure 5 isolates just the $O(n \log n)$ algorithms on random data so we can actually see the differences between them (they get diminished in the other charts by the $O(n^2)$ curves).

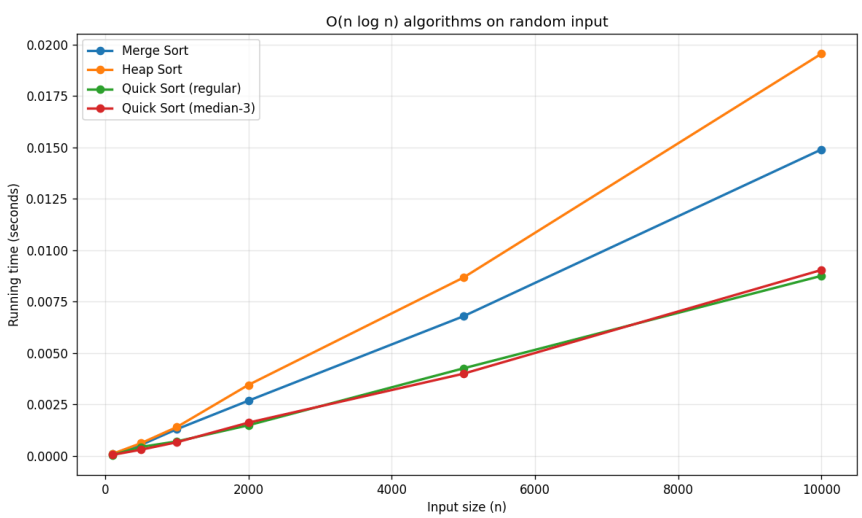


Figure 5: $O(n \log n)$ algorithms on random input

Quick Sort (both versions) is the fastest, followed by Merge Sort, then Heap Sort. This makes sense because quicksort has better cache locality and lower constant factors.

5.6 Quick Sort Comparison

Figure 6 compares regular quicksort against the median-of-three version across different distributions. The difference is dramatic on sorted and reverse input, regular quicksort goes $O(n^2)$ while median-of-three stays $O(n \log n)$.

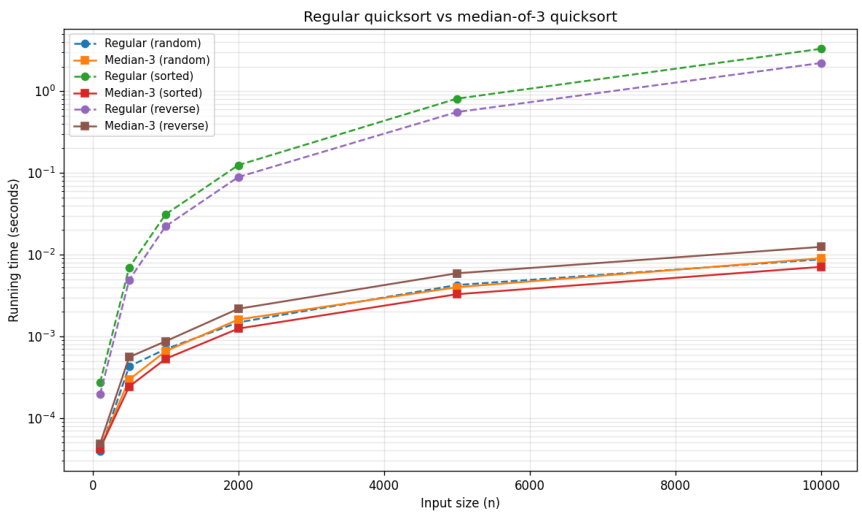


Figure 6: Regular quicksort vs median-of-three quicksort

6 Reflection

6.1 Do the results match the theory?

Yes. The $O(n^2)$ algorithms clearly show quadratic growth in the charts and the $O(n \log n)$ ones grow much more slowly.
Some specifics:

- Bubble sort on reverse input at $n = 10{,}000$ takes about 4.6 seconds, while at $n = 5{,}000$ it takes about 1.1 seconds. $10{,}000^2/5{,}000^2 = 4$, and $4.6/1.1 \approx 4.2$, which is close to the expected 4x ratio.
- Merge sort at $n = 10{,}000$ takes 0.015s and at $n = 5{,}000$ takes 0.007s. The ratio is about 2.1x, consistent with $n \log n$ growth.

6.2 At what point does $O(n^2)$ become impractical?

Based on the data, $O(n^2)$ algorithms start becoming noticeably slow around $n = 2{,}000$ (taking over 100ms) and are genuinely painful at $n = 10{,}000$ (multiple seconds). For any real application with more than a few thousand elements, I would want to use an $O(n \log n)$ algorithm.

6.3 When and what algorithm to use?

- **General purpose:** Quick Sort (median-of-three), fastest on average and doesn't have the worst-case problem of regular quicksort
- **Nearly-sorted data:** Insertion Sort, hard to beat when data is almost sorted, basically $O(n)$
- **Need stability:** Merge Sort, it is stable and has guaranteed $O(n \log n)$, at the cost of extra memory
- **Avoid:** Regular quicksort on data that might be sorted or nearly sorted

6.4 Merge Sort vs Quick Sort

On random data, both quicksorts are faster than merge sort (8–9ms vs 15ms at $n = 10{,}000$). This is because quicksort does the sorting in-place and has better cache performance. However, merge sort has the advantage of being stable and having guaranteed $O(n \log n)$ worst case, which quicksort does not have (even with median-of-three, adversarial inputs could still cause $O(n^2)$ in theory).

7 User Interface

I built a GUI using Tkinter with an embedded Matplotlib chart. The GUI lets you:

- Select one or more algorithms via checkboxes
- Set the input size
- Choose the input distribution (random, sorted, reverse, nearly-sorted)
- Run the benchmark and see results in a table and bar chart

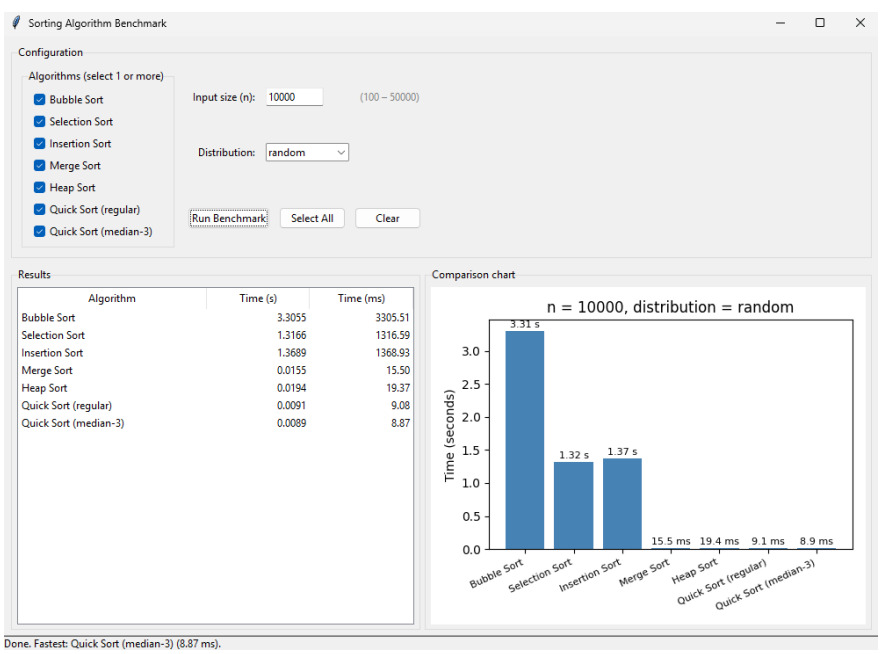


Figure 7: Tkinter GUI

The benchmark runs on a background thread so the window doesn't freeze. I originally ran it on the main thread and the whole GUI would lock up until the sorting finished, so I had to move it to a separate thread using Python's `threading` module.

I also started working on an animated visualization of the sorting process using Matplotlib's `FuncAnimation` but couldn't get it to work properly with Tkinter's event loop, it kept crashing with threading errors and the bars wouldn't update correctly.

8 Conclusion

The benchmarks confirm that the theoretical time complexities hold up in practice. The $O(n^2)$ algorithms (bubble, selection, insertion) grow quadratically and become impractical past a few thousand elements, while the $O(n \log n)$ algorithms (merge, heap, quicksort) scale well. The median-of-three optimization for quicksort makes a huge difference on sorted and reverse-sorted input, turning a 3+ second runtime into under 15 milliseconds. For general use, quicksort with median-of-three pivot selection should be the best choice.